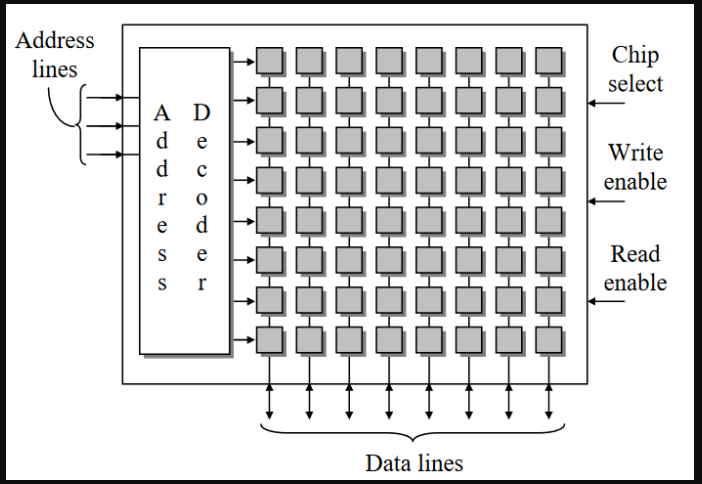
Lecture 8
Topics: I/O

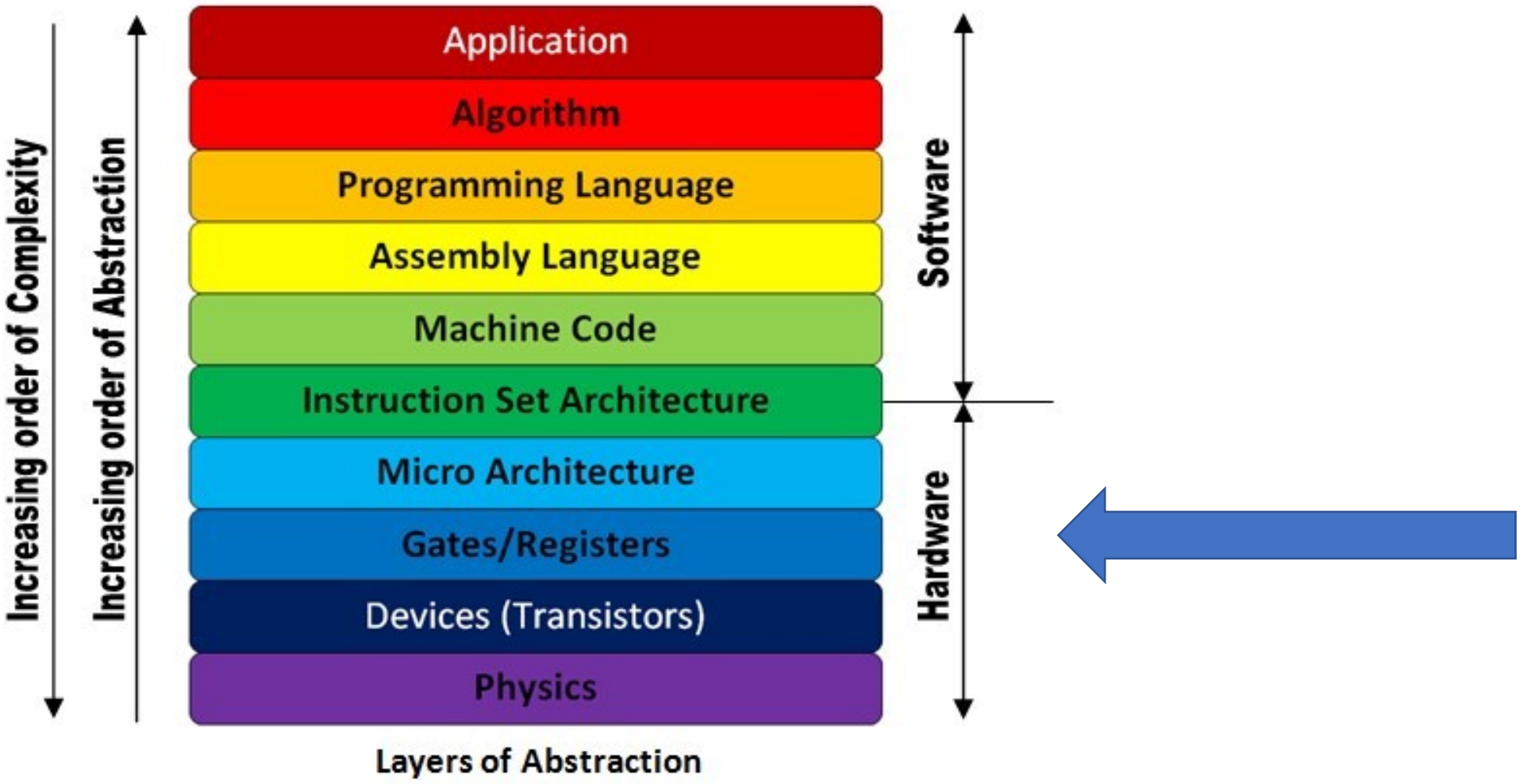


Probir Roy
Assistant professor, CIS

CIS-310: Computer Organization & Assembly Language

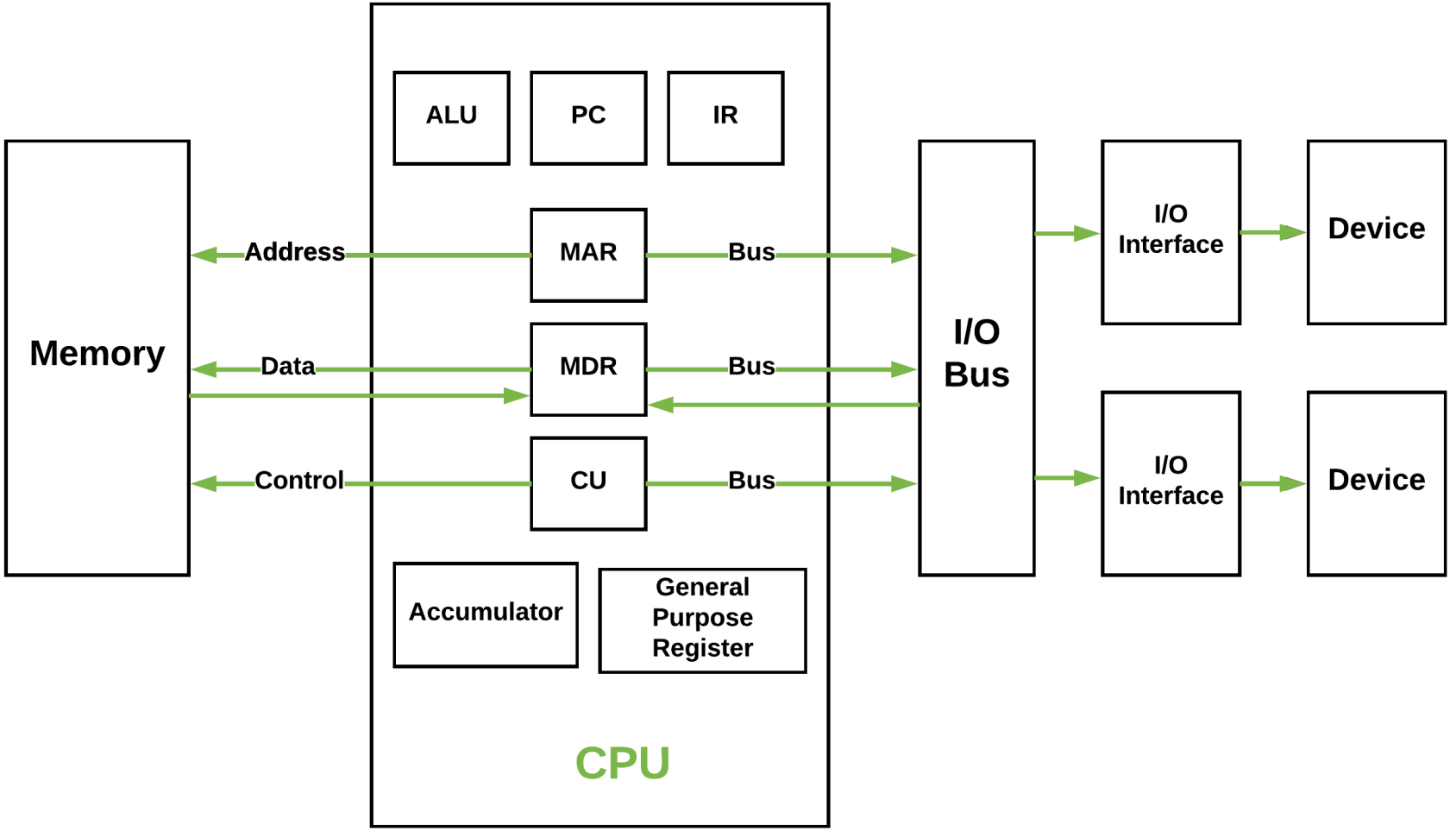
MW

DEARBORN

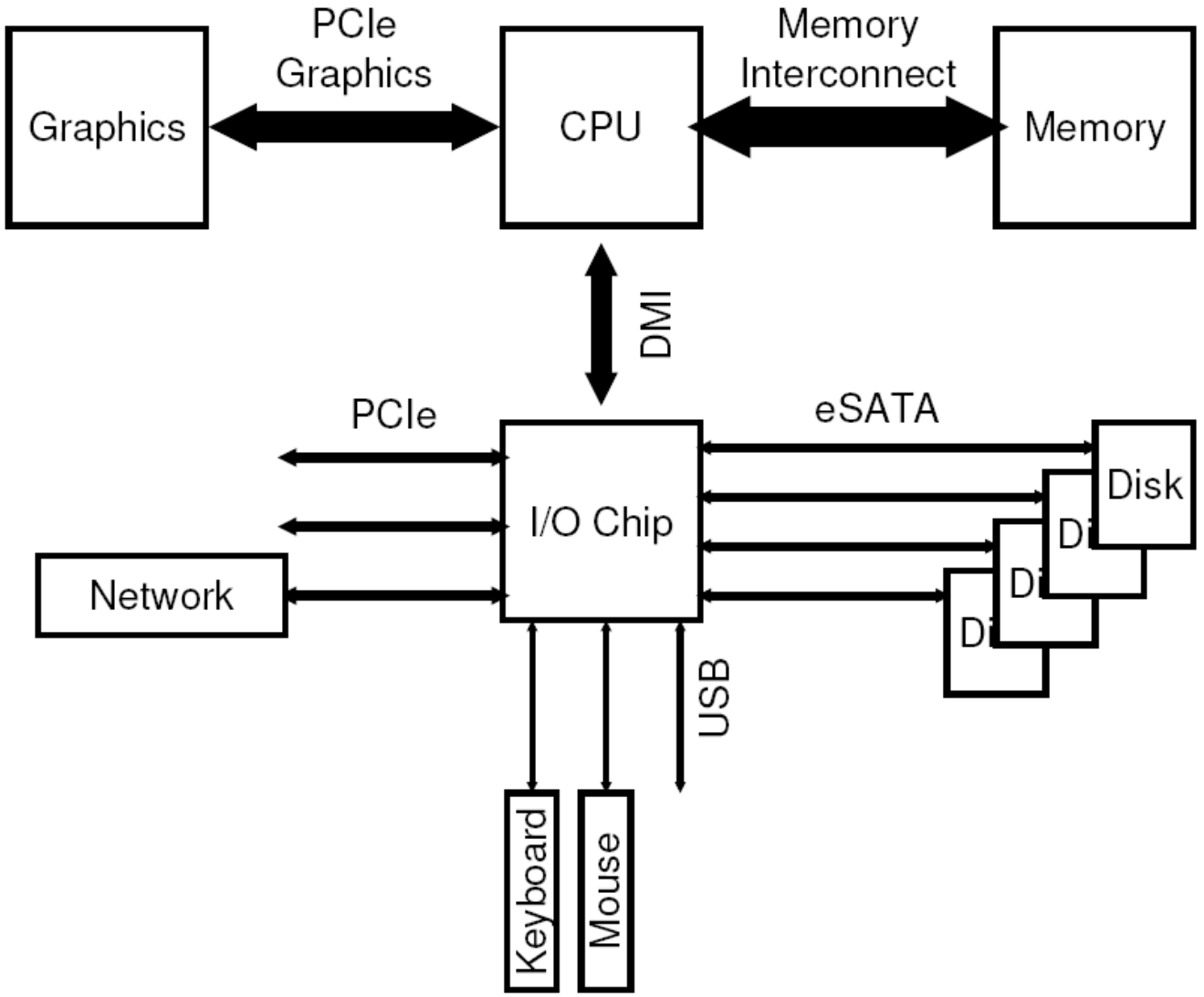


https://faculty.etsu.edu/tarnoff/ntes2150/Ch12_v02.pdf

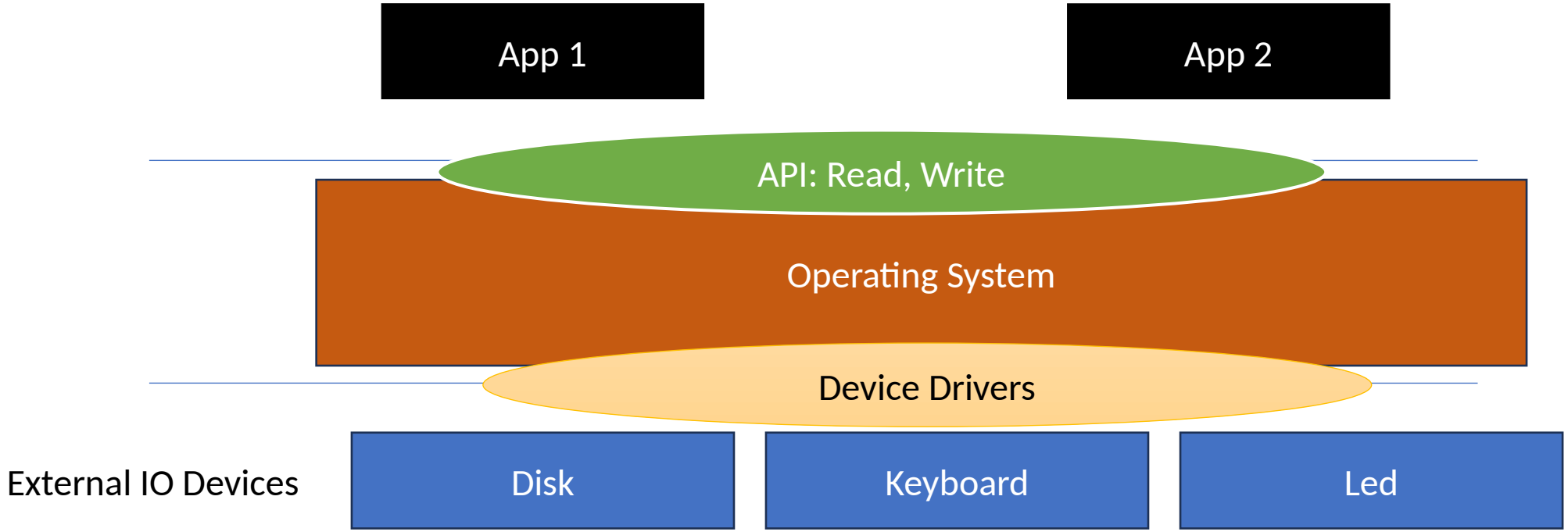
Computer architecture



Modern System Architecture



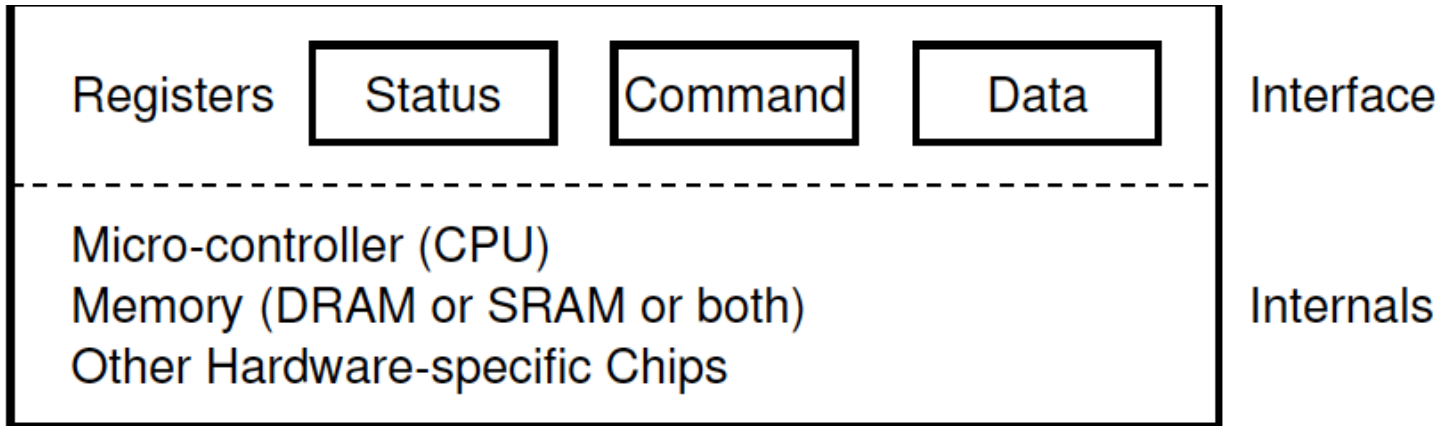
IO device management



Outline

- System architecture
- I/O protocols

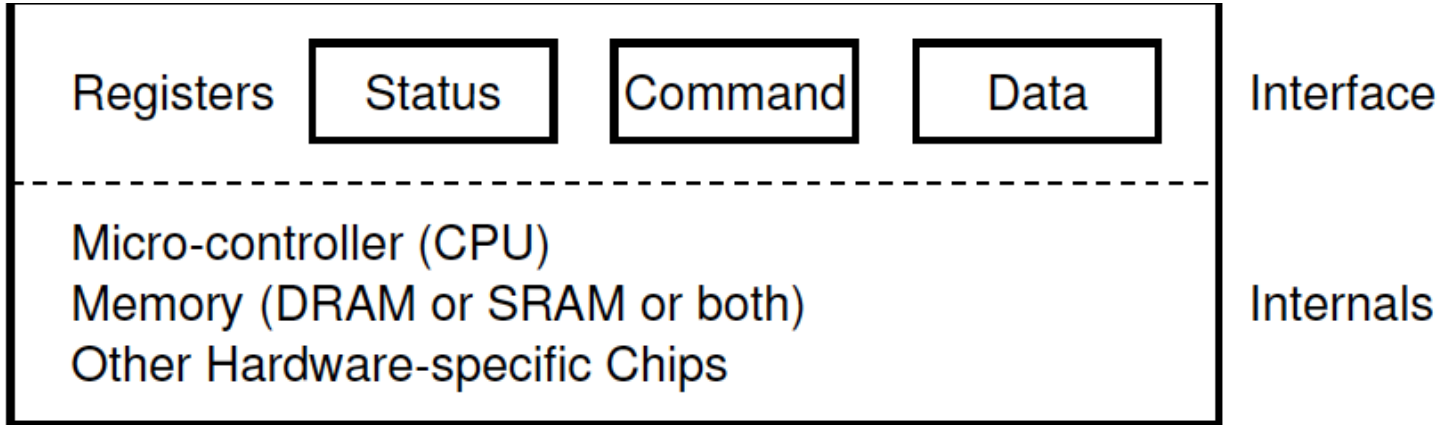
A canonical device (cont'd)



- Hardware interface
  - It allows the system software to control its operation
- Three registers:
  - a status register, which can be read to see the current status of the device;
  - a command register, to tell the device to perform a certain task;
  - a data register to pass data to the device, or get data from the device.

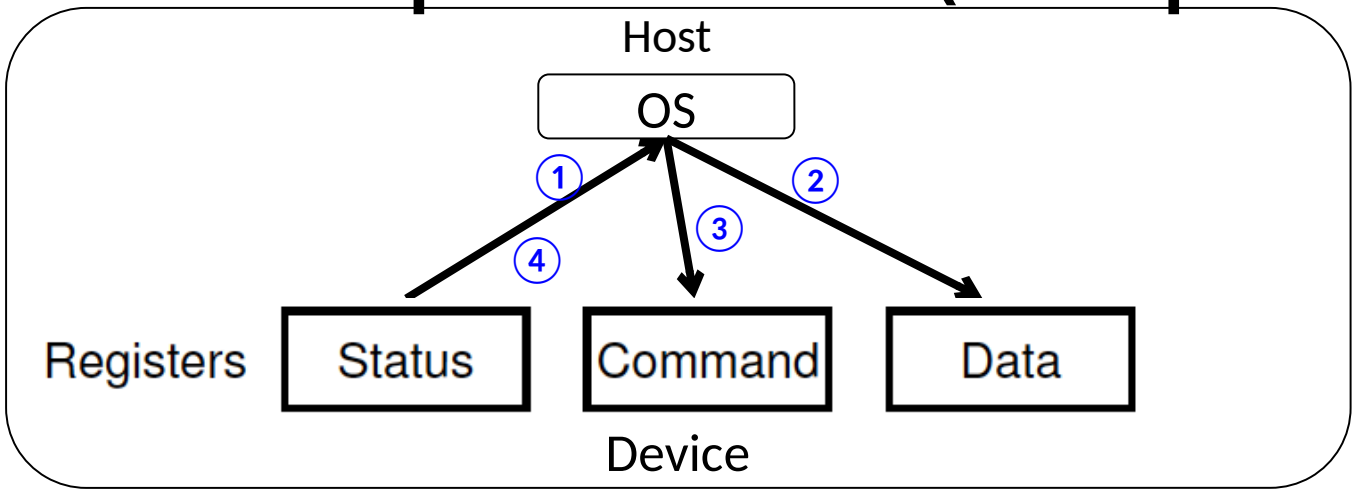
By reading and writing these registers, the OS can control device behavior.

A canonical device (cont'd)



- Internal structure
  - It is implementation specific and is responsible for implementing the abstraction (interface) the device presents to the system.
  - Firmware (i.e., software within a hardware device)

A canonical protocol (output)



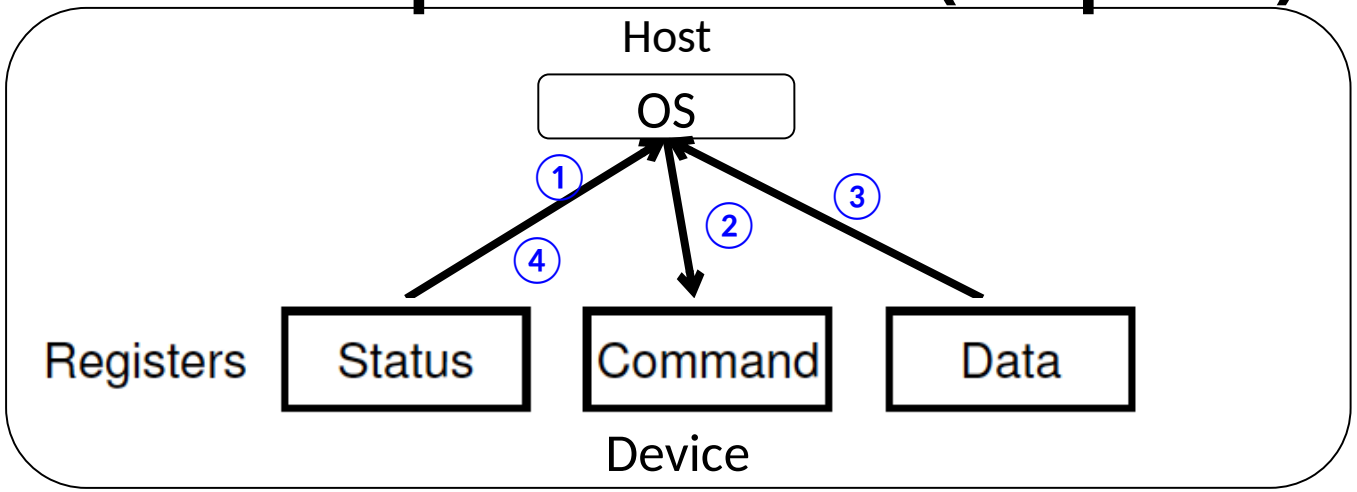
① While (STATUS == BUSY) {;}
(wait until device is not busy - polling)

② Write data to DATA register
(e.g., multiple writes need to take place to transfer a disk block to the device)

③ Write command to COMMAND register
(lets the device know that both the data is present and that it should begin working on the command)

④ While (STATUS == BUSY) {;}
(wait until device is done with your request - polling)

A canonical protocol (input)



① While (STATUS == BUSY) {;}
(wait until device is not busy - polling)

② Write command to COMMAND register
(lets the device know that it should begin working on the command)

③ Read data from DATA register
(transfer a disk block to memory)

④ While (STATUS == BUSY) {;}
(wait until device is done with your request – polling)

Problems in the canonical protocol

- Polling seems inefficient.
  - It wastes a great deal of CPU time just polling the (potentially slow) device, instead of switching to another ready process.
  - Like the spin lock.
- Solution to spin lock
  - We put the calling thread to sleep for locking.
  - Wake up the sleep thread for unlocking.
- Analogy
  - Check email constantly, or
  - Through email notification

Outline

- System architecture
- I/O protocols
  - Polling
  - Interrupts

Interrupts

- No waiting but sleep:
  - The OS can issue a request, put the calling process to sleep, and context switch to another process.
- Notification:
  - When the device is finally finished with the operation, it will raise a hardware interrupt, causing the CPU to jump into the OS at a predetermined interrupt service routine (ISR) or more simply an interrupt handler.
  - The handler is just a piece of OS code that will finish the request (for example, by reading data and perhaps an error code from the device) and wake the process waiting for the I/O.

Benefit

- Interrupts thus allow for overlap of computation and I/O, which is key for improved utilization.

1: process 1
2: process 2
p: polling

Polling

```
CPU    1111111111ppppppppppp111111111111
Disk   ----------11111111111------------
```

Interrupt

```
CPU    1111111111222222222211111111111
Disk   ----------11111111111------------
```

Interrupts is not always the best solution

- Using an interrupt for a fast device will slow down the system:
  - Switching to another process, handling the interrupt, and switching back to the issuing process is expensive.
  - It may be best to poll.
- There are also cases where a flood of interrupts may overload a system and lead it to livelock:
  - Find itself only processing interrupts and never allowing a user-level process to run and actually service the requests (like thrashing).

Challenge with Synchronous Programming model

- The Challenge with Synchronous I/O:
  - Even with interrupts allowing the OS to run other processes, the process that initiates the I/O remains blocked, waiting for completion.
  - This waiting can cause the initiating process to starve, leading to inefficient CPU utilization and reduced responsiveness.
- Why Interrupts Alone Aren’t Enough:
  - Although interrupts notify the system when an I/O operation is ready, the initiating process still adheres to a synchronous model, halting its own progress.
  - The synchronous nature forces the process to idle, limiting its ability to perform other useful work during I/O delays.

Motivation for Asynchronous Programming

- Decouples the start of an I/O operation from its completion.
  - Allows the process to continue executing other tasks rather than being blocked.
  - Improves system responsiveness and resource utilization by ensuring the process is not starved while waiting for I/O completion.

Introducing Asynchronous Programming

- Core Idea:
  - Initiate an I/O operation and continue executing other tasks immediately—no waiting.
- How It Works:
  - Uses callbacks, promises, or async/await mechanisms to handle I/O completions.
  - The underlying system still leverages interrupts to notify about completion, but this complexity is hidden from the programmer.
- Smooth Transition:
  - Asynchronous programming builds naturally on the interrupt model by providing a higher-level abstraction that manages I/O without blocking the main execution flow.

How Asynchronous Programming Operates

- Behind the Scenes:
  - The OS and drivers continue using interrupts to signal when data is ready.
  - An asynchronous framework (or runtime) catches these signals and triggers the corresponding callback or resumes a suspended task.
- Program Flow Example:
  - Initiate an I/O request → Continue processing other tasks → Receive an event (or callback) when the operation is complete → Process the results.
- Outcome:
  - Better resource utilization and a more responsive system design.

Advantages of Asynchronous Programming

- Improved CPU Utilization:
  - Overlap computation with I/O tasks rather than waiting idly.
- Enhanced Responsiveness:
  - Keeps applications interactive by not blocking the main thread.
- Scalability:
  - Efficiently handles multiple concurrent I/O operations, which is critical in modern systems.
- Abstraction & Simplicity:
  - Hides the low-level details of interrupts and context switching, allowing developers to write clearer, more maintainable code.

Conclusion

- **Evolution Recap:**
  - **Polling:** A simple but inefficient start.
  - **Interrupts:** Improved efficiency through hardware notifications, but with overhead.
  - **Asynchronous Programming:** The next step—abstracting the complexity of interrupts to deliver non-blocking, concurrent I/O operations.

Code example

- https://github.com/proywm/Arduino-Asynchronous-Programming
- https://github.com/proywm/ArduinoNano33BLE_Temp/tree/main